WHY TRAFFIC?

Forecasting highway speed 15, 30 and 60 minutes ahead from sensor data · Phase 1 · Team T082, Section B

t + 60 min

Real-world impact

A traffic-signal controller needs the next hour's jam map, not the current one. Phase 1 builds that forecast; Phase 2 will use it.

207 sensors

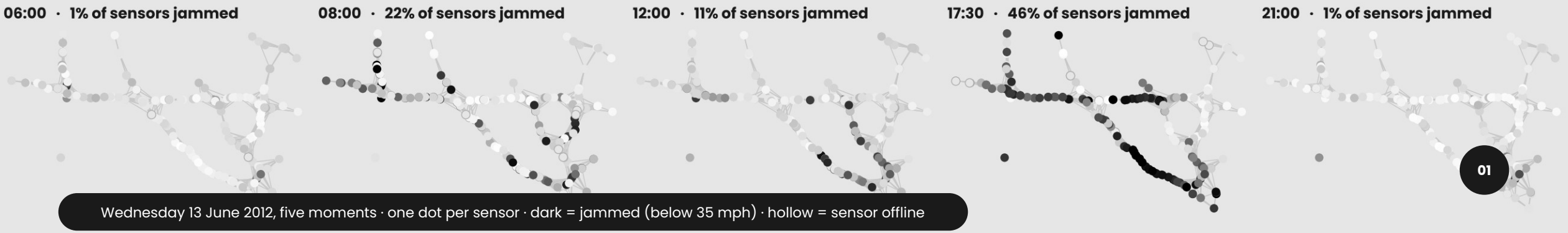
Real, public data

METR-LA: a speed reading every 5 minutes on Los Angeles highways, March–June 2012. The standard benchmark (Li et al., 2018).

15–20 km/h

A physics hint

A jam's tail moves backwards against the cars (LWR model, 1955), so the sensor ahead of you knows your near future.



Wednesday 13 June 2012, five moments · one dot per sensor · dark = jammed (below 35 mph) · hollow = sensor offline

THE DATA

METR-LA is one table: a row every five minutes from 1 March to 27 June 2012, a column for each of 207 sensors, and in each cell the average speed in mph. It comes with a road map that says how far apart the sensors are by road, so we know which sensors are neighbours.

17 WEEKS



- **12** train · 1 March – 23 May
- **2** validation · 23 May – 4 June
- **3** test · 4 – 27 June

Split by date, never shuffled, with a one-hour gap at each boundary: a test answer never appears in training. The validation weeks only decide when to stop training.

HOW WE GOT THE EXAMPLES

STEP	TRAIN	VALIDATION	TEST
1 Five-minute time slots	23,967 1 Mar – 23 May	3,403 23 May – 4 Jun	6,831 4 – 27 Jun
2 Readings (slots × 207 sensors)	4,961,169	704,421	1,414,017
3 Exactly 0 mph → marked unknown	-353,732 7.1%	-49,317 7.0%	-171,637 12.1%
4 Inputs per example: 34 numbers 26 about the sensor itself + 8 about its neighbours			
5 Examples whose answer (speed +60 min) is known	4,607,444	655,115	1,242,308
Examples used, per horizon of which jammed (below 35 mph)	**4.6 M** 8.6%	**0.66 M** 8.7%	**1.24 M** 10.3%

A reading of exactly 0 mph is an offline sensor, not stopped traffic: a highway never averages zero over five minutes, the zeros come in runs, and on 6% of slots all 207 sensors read 0 at once. We mark them unknown — nothing deleted, nothing invented. Tree models accept unknown inputs; an unknown answer is skipped in training and in scoring.

THE INPUTS

What is local / spatial?

Local = the 26 numbers a sensor knows about itself. Spatial = those plus 8 numbers about its neighbours. Every model is trained both ways, so the difference measures exactly what the neighbours are worth.

How are the groups made?

By what the number describes. Nothing from the future leaks in: lags look backwards, the usual speed is computed from training weeks only.

- **A** Own last hour — 12 numbers
- **B** Summary of that hour — 6 numbers
- **C** The clock — 6 numbers
- **D** Usual speed — 2 numbers
- **E** Neighbours — 8 numbers

GRP	FEATURE	WHAT IT TELLS THE MODEL	COUNT	USED BY	IMPOR.
A	**lag_0 … lag_11**	speed now and in the previous 11 slots (one hour)	12	L + S	34%
B	**win_mean / std / min / max**	summary of that hour	4	L + S	12%
B	**win_frac_missing, trend_15min**	share of the hour unknown; change over 15 min	2	L + S	2%
C	**tod_sin / cos, dow_sin / cos**	time of day, day of week — midnight wraps round	4	L + S	10%
C	**is_weekend, hour**	weekend flag; hour of the day	2	L + S	8%
D	**ha_now, dev_from_ha**	usual speed at this time of week (train only); gap from usual	2	L + S	9%
E	**up_mean (now, -5, -10 min)**	upstream neighbours: what is about to arrive	3	S	8%
E	**down_mean (now, -5, -10 min)**	**downstream neighbours: a queue growing back to you**	3	S	10%
E	**nbr_min, nbr_frac_congested**	slowest neighbour; share of neighbours jammed	2	S	7%
—	**speed +15 / +30 / +60 min**	the answers we predict	3	—	—

Importance = share of XGBoost's split gain, averaged over all 207 sensors and 3 horizons. L = local models, S = spatial models. Neighbour weights follow the road graph: nearer neighbours count more (Gaussian kernel on road distance, as in DCRNN).

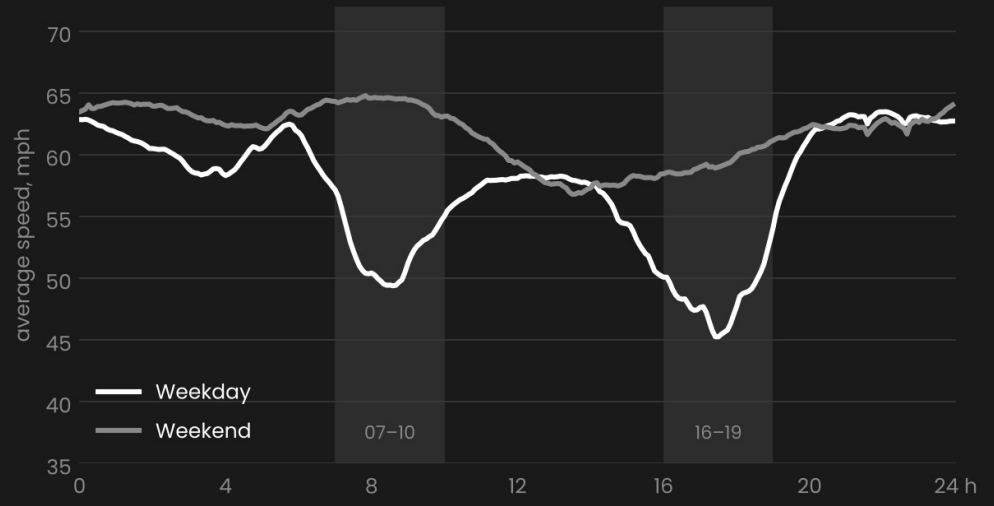
EDA

Exploratory data analysis, all four months

AVERAGE VALUE	OFF-PEAK	RUSH HOUR
Mean speed (mph)	59.6	50.1
Spread of speeds, std (mph)	10.6	18.5
Readings below 35 mph	5.2%	26.5%
Sensors jammed at 17:00, weekdays	**—**	**35%**

Rush hour = weekdays 07–10 and 16–19. Weekends look like off-peak all day (mean 61.7 mph). The same pattern repeats every week, so "the usual speed at this time of week" becomes an input, and the models are scored in rush hour and off-peak separately.

AVERAGE SPEED BY TIME OF DAY · ALL FOUR MONTHS

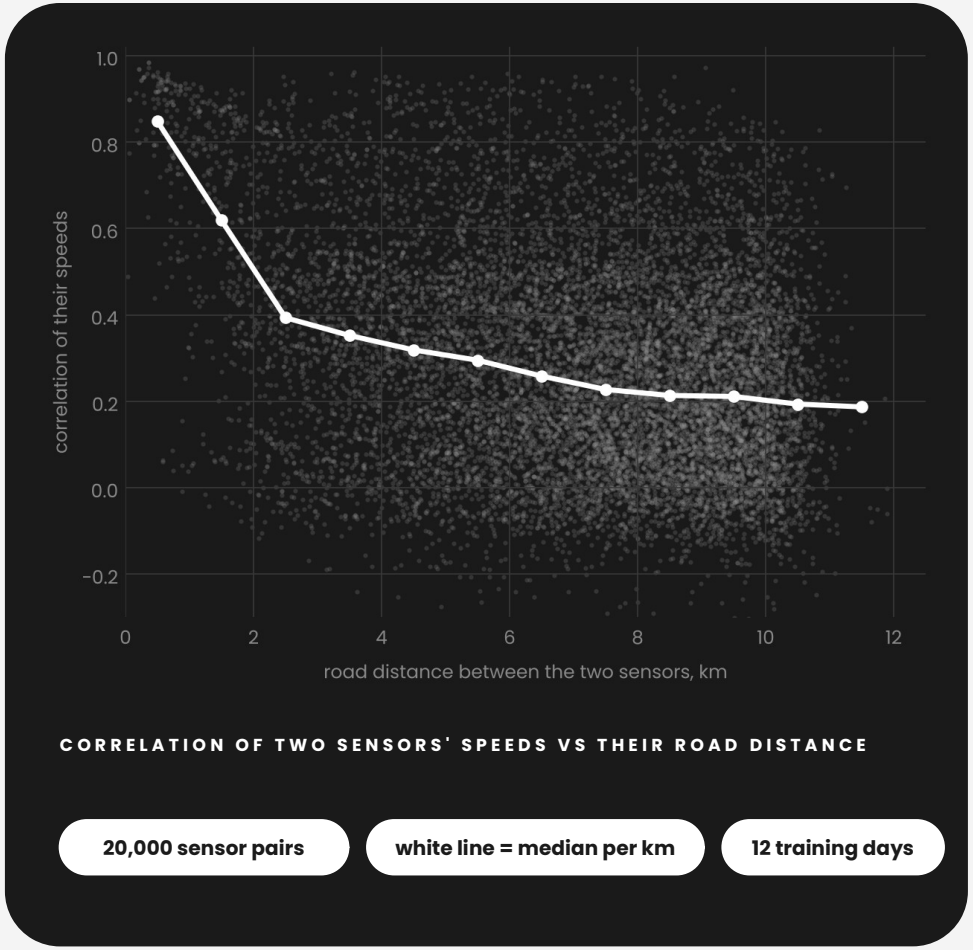


35% of sensors are jammed at 17:00 on a weekday; about 24% at 08:00, under 1% at 03:00

.84 correlation of a sensor's speed with its own speed one hour earlier → the last hour is a strong input

.64 with 24 hours earlier → a daily cycle worth a feature

Two slow periods on weekdays, the evening one deeper; weekends flat. The shaded bands are the rush-hour windows used in the evaluation.



CORRELATION OF TWO SENSORS' SPEEDS VS THEIR ROAD DISTANCE

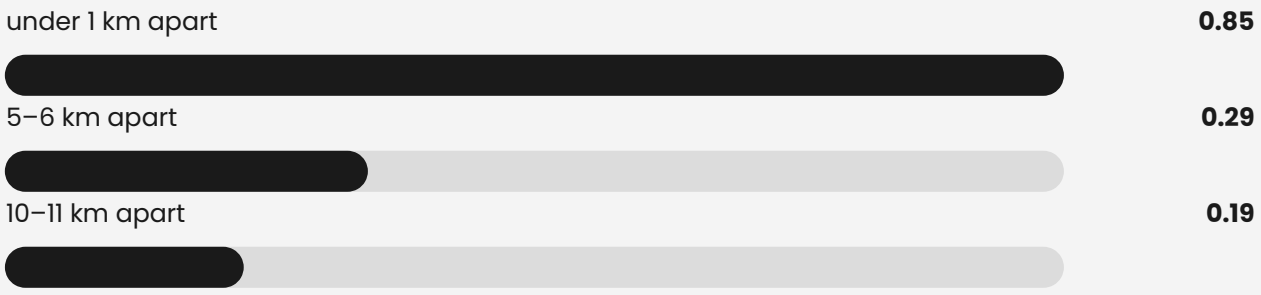
20,000 sensor pairs | **white line = median per km** | **12 training days**

WHY NEIGHBOURS

A jam grows like a queue: cars join at the back, so its tail moves against the traffic at 15–20 km/h (Lighthill–Whitham–Richards model, 1955). The sensor ahead of you sees your next few minutes first. We turned that into eight numbers.

up_mean (now, -5, -10 min)	upstream neighbours: what is about to arrive
down_mean (now, -5, -10 min)	**downstream neighbours: a queue growing back to you**
nbr_min, nbr_frac_congested	slowest neighbour; share of neighbours jammed
weights	nearer neighbours count more; unknown neighbours left out

CORRELATION WITH A NEIGHBOUR'S SPEED, BY ROAD DISTANCE



Only close neighbours carry information, so one-hop features are the right first step. A wave at 15–20 km/h crosses one hop (1–3 km) in 5–15 minutes, so they should help most at short horizons — keep that in mind for the results.

01 · FLOOR

Hist. average

The usual speed at this time of week. No learning, no present.

VAL MAE 15	VAL MAE 60	
4.32	4.30	no present

02 · FLOOR

Persistence

The speed stays what it is now. No learning, no memory.

VAL MAE 15	VAL MAE 60	
3.21	4.75	no memory

03 · LINEAR

Ridge

A weighted sum of the 26 inputs. Is a straight line enough?

VAL MAE 15	VAL MAE 60	
3.10	4.87	straight lines

04 · MAIN MODEL

XGBoost

Trees built one after another, each fixing the last one's mistakes. 621 models.

VAL MAE 15	VAL MAE 60	
2.70	3.63	best → test

05 · CROSS-CHECK

Random Forest

100 trees averaged; one forest predicts all three horizons.

VAL MAE 15	VAL MAE 60	
2.73	3.69	a tie

THE MODELS

Validation weeks only (23 May – 4 June), never the test set; MAE in mph at 15 and 60 minutes ahead. Each model answers one question, from a no-learning rule to boosted trees. One model per sensor; XGBoost one per sensor and horizon — 621 in all, about 12 minutes on two CPU cores.

Why XGBoost

Trees combine clues ("5 pm and speed dropping and the sensor ahead is slow"), which a straight line cannot: it beats Ridge by 22% at 60 min. It accepts unknown inputs, so the outages need no filling. It stops adding trees when the validation error stops improving — about 130 of the 600 allowed.

But it is a tie

The Random Forest is 0.03–0.06 mph behind on validation and 0.01 on test, with a slightly lower RMSE. XGBoost was chosen because it trains three times faster, not because it is clearly better.

TEST SET · XGBOOST + NEIGHBOURS · 60 MIN · JAM = BELOW 35 MPH

	ACTUAL JAM	ACTUAL FREE
PREDICTED JAM	78,082 jams caught	13,604 false alarms
PREDICTED FREE	50,475 jams missed	1,100,147 correct

MODEL ACCURACY
94.8%

ALWAYS "FREE"
89.7%

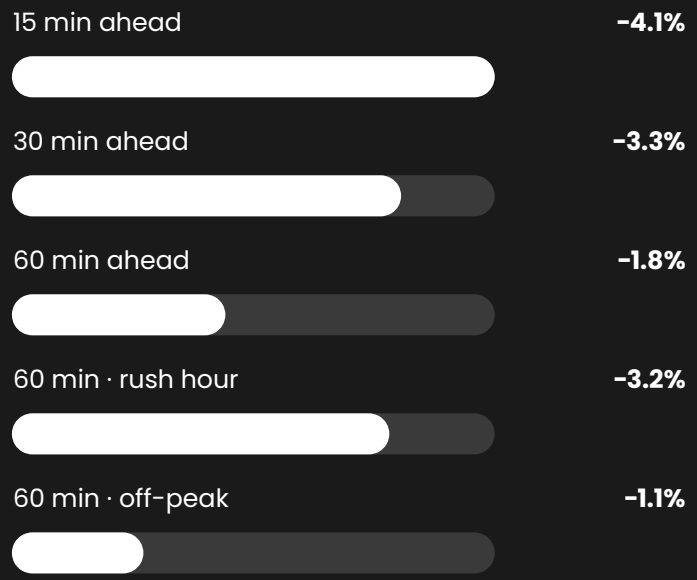
PRECISION 0.85 · RECALL 0.61 · F1 0.71

THE RESULTS

TEST MAE, MPH	XGB + NBRS	PERSISTENCE	HIST. AVG
15 min ahead	**2.87**	3.49	4.19
30 min ahead	**3.32**	4.18	4.19
60 min ahead	**3.78**	5.33	4.19
RMSE at 60 min	**7.07**	10.27	7.86

Why not accuracy: 90% of answers are "free", so always saying "free" scores 89.7%. MAE — the average mistake in mph, counting only cells where the true speed is known — cannot be fooled that way. Persistence is excellent at 15 min and collapses by 60; the historical average ignores the present; the learned model combines both. XGB-local is 22% better than Ridge at 60 min; the Random Forest is 0.01 mph behind. Test weeks (4–27 June) were scored once, at the end. DCRNN, the published deep graph model on this data, reports 3.60 at 60 min.

SAME XGBOOST, WITH VS WITHOUT 8 NEIGHBOUR NUMBERS · MAE REDUCTION



147 of 207 sensors improve at 60 min (up to 7.2%). The Random Forest shows the same: -3.4 / -3.1 / -1.4%. Same rows, same split, same settings — the only change is the eight neighbour columns.

KEY INSIGHTS

Neighbours help — most in rush hour

Adding the eight neighbour numbers lowers the error at every horizon, three times more in the peaks: congestion really does spread between sensors. The model's second most-used input is the downstream neighbours.

The help fades at 60 minutes

One hop sees 5–15 minutes of a jam moving at 15–20 km/h; what hits you in an hour is still several sensors away. That fade is the gap for Phase 2's graph network, which looks several hops out.

Rush hour is the hard part

Error doubles in the peaks (5.67 vs 3.31 mph at 60 min) and sits almost entirely below 35 mph. Recall 0.61: a jam that has not arrived yet is invisible in a sensor's own history.

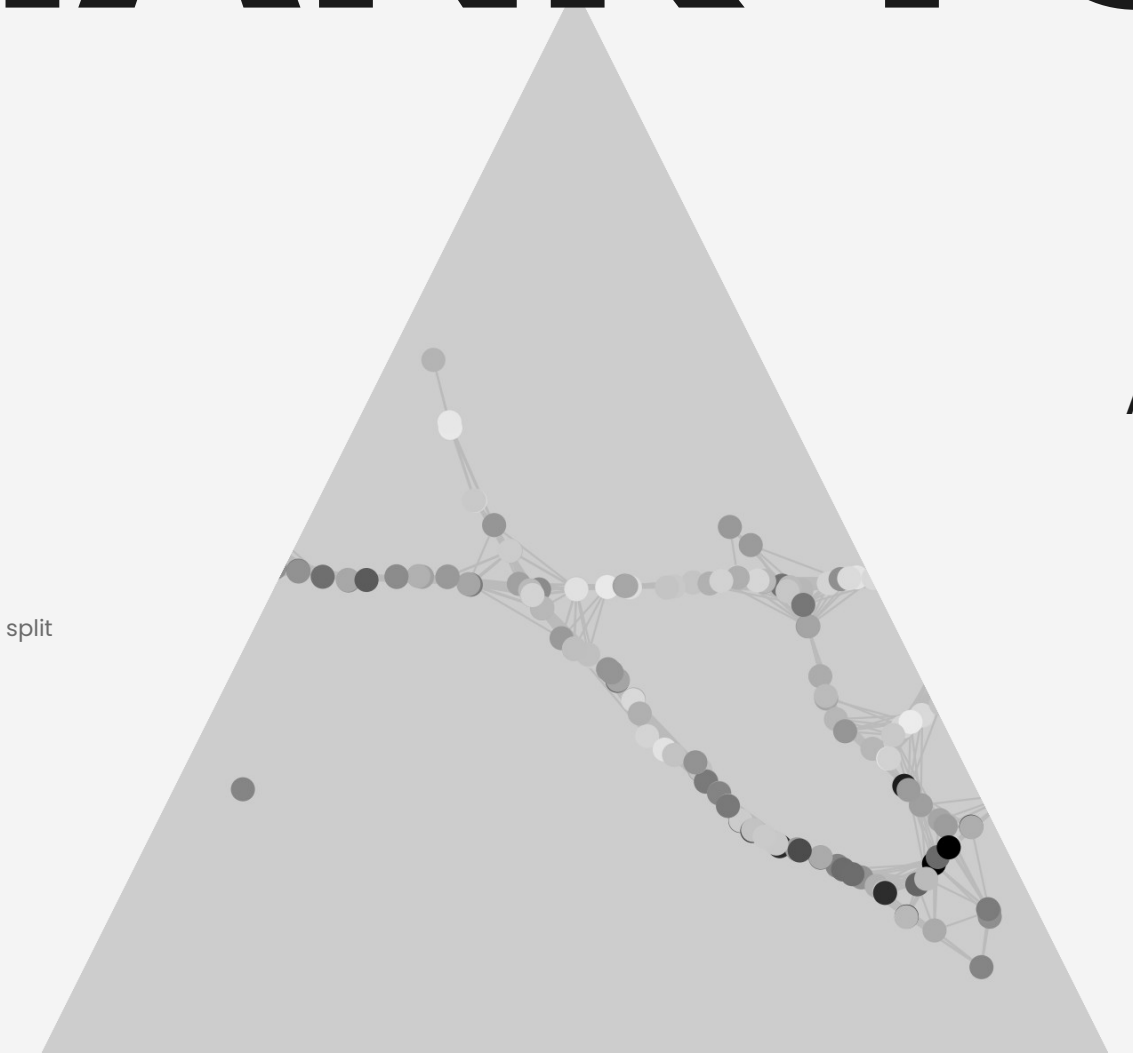
The numbers are honest

Split by date with a gap; settings fixed in advance; validation ranks the models exactly like the test set; early stopping kept ~130 of 600 trees. One command reruns everything; 8 tests check split, features and scoring.

Next steps

Phase 2: a Graph Attention Network on the same data, split and scoring; check whether its attention follows the backward wave; any signal-control experiment is a simulation only.

THANK YOU



Presented by

Team T082 · Section B

Kartik Madaan
Data pipeline, road graph, repository, EDA

Arhan Alam
Feature engineering, LWR neighbour features, time split

Chirag Lalwani
Model training: XGBoost, Random Forest, Ridge

Phase 1

Advanced ML & Deep Learning (CSAI)

Siddhanth Sadshiv Raikar
Models and the IEEE report

Abhijeet Dey
Evaluation: masked metrics, ablation, results

Code
github.com/madaankartik/traffic-aml-project